

VVS Assignment 2: Comprehensive Testing of the Calendar Application

Rodrigo Neto

June 16, 2026

1 Introduction

This report describes the testing strategy and implementation for the *Calendar* application. The goal was to provide a robust test suite covering multiple levels of the application: Unit Testing, Integration Testing (3rd party, REST, and Database), and End-to-End (E2E) testing, along with Continuous Integration setup.

2 Step-by-Step Solution

The following steps were taken to fulfill the assignment requirements:

1. **Environment Setup:** Added Maven dependencies for JUnit 5, Mockito, Selenium, WebDriverManager, and Spring Security Test.
2. **SUT Adaptation:** Modified the source code to improve testability, specifically injecting `RestClient.Builder` in discovery providers and adding `setId` methods to model classes.
3. **Unit and Model Testing:** Developed unit tests for services using Mockito and domain models to isolate and verify business logic.
4. **3rd Party Integration:** Used `@RestClientTest` to mock external API responses and verify parsing logic.
5. **REST API Integration:** Expanded test coverage to all controllers (`AuthController`, `MeetingController`, `DiscoveryController`, `CalendarController`, `ICalController`) using `MockMvc` and `@WebMvcTest`.
6. **Security Testing:** Implemented a dual-layer security testing strategy: (1) a fast `@WebMvcTest` unit test for authorization rule regression, and (2) a full `@SpringBootTest` integration test for end-to-end security verification.
7. **Database and Repository Integration:** Verified the JPA persistence layer and custom repository query methods using `@SpringBootTest` and `@DataJpaTest` with an H2 database.
8. **End-to-End Testing:** Created multiple Selenium journeys to verify full user flows, including registration, login, meeting proposal, discovery, logout, iCal feed generation, and meeting invitation responses.
9. **CI/CD Optimization:** Categorized all tests using JUnit 5 tags and configured a multi-tier GitHub Actions pipeline with manual triggers, concurrency controls, and artifact management.
10. **Bug Discovery and Fixes:** Through comprehensive testing, identified and some bugs in the application (detailed in Section 6).

3 Testing Details

3.1 Unit and Model Testing (Business Logic)

Targeting the *model*, *service*, and *discover* packages.

- **UserTest**: Unit tests for the **User** entity constructor (verifying auto-generation of unique iCal tokens) and basic getters/setters.
- **MeetingTest**: Unit tests for the **Meeting** entity, specifically validating the `isConfirmed()` logic across various participant invitation status configurations (empty, all accepted, pending, and declined). **Note**: The original file `MeetingTest.java` incorrectly contained `MeetingParticipantTest` class; this was fixed by creating a proper `MeetingTest` and renaming the original to `MeetingParticipantTest.java`.
- **MeetingParticipantTest**: Unit tests for the **MeetingParticipant** entity, verifying constructor field initialization, status transitions (PENDING → ACCEPTED/DECLINED), and getter correctness. (Renamed from `MeetingTest.java`.)
- **InviteStatusTest**: Defensive enum tests ensuring the three invitation states (PENDING, ACCEPTED, DECLINED) are stable and correctly ordered.
- **UserServiceTest**: Tests user registration (success, duplicate username, empty username boundary, null password boundary) and user retrieval.
- **MeetingServiceTest**: Tests proposing meetings (including edge cases such as null title, null organizer, empty invitee list, duplicate invitees, and self-invitation prevention), responding to invites (verifying exceptions when invitation is not found or status is null), and copying events from discovery (testing default and explicit end-time handling).
- **AppUserDetailsServiceTest**: Ensures Spring Security correctly loads user details.
- **ICalServiceTest**: Verifies iCal feed generation, special character escaping, VEVENT field layout, and CONFIRMED vs TENTATIVE status mapping.
- **DiscoveryServiceTest**: Tests event aggregation, de-duplication, sorting from multiple providers, partial failure tolerance, empty query handling, and unconfigured provider filtering.
- **AgendaLxProviderTest**: Pure Mockito unit test (no Spring context) mocking the full `RestClient` fluent API to verify HTML parsing, time fallback logic, description truncation, venue extraction, and exception swallowing.

3.2 Integration Testing (3rd Party Sources)

Targeting the *discover* package using `MockRestServiceServer`.

- **AgendaLxProviderIntegrationTest**: Verifies parsing of Lisbon's Agenda Cultural JSON via `@RestClientTest`.
- **TicketmasterProviderIntegrationTest**: Verifies parsing of Ticketmaster API responses via `@RestClientTest`.
- **SeatGeekProviderIntegrationTest**: Verifies parsing of SeatGeek API responses via `@RestClientTest`.

3.3 REST API Integration (Controllers)

Comprehensive coverage using `MockMvc`.

- **AuthControllerTest**: Tests registration, login flows, and root redirect (`/` → `/calendar`).
- **MeetingControllerTest**: Tests meeting proposal, error handling, and response to invitations (accept/decline actions).
- **DiscoveryControllerTest**: Tests event discovery results and copying discovered events.
- **CalendarControllerTest**: Tests calendar rendering for authenticated users, empty calendar handling, and unauthenticated redirect.
- **ICalControllerTest**: Verifies the public iCal feed endpoint (valid token, empty calendar, invalid token 404).

3.4 Security Testing (Dual-Layer Strategy)

- **SecurityConfigTest** (`@WebMvcTest`, unit/smoke): Fast, isolated verification that public endpoints are accessible, protected endpoints redirect to login, H2 console is permitted, CSRF is correctly disabled for `/ical` and `/h2-console`, and logout invalidates the session. Runs in milliseconds.
- **SecurityTest** (`@SpringBootTest`, smoke): Full integration verification that public endpoints are accessible and protected endpoints redirect to login for anonymous users, while allowing access to authenticated users. Uses the real application context and database.

This dual-layer approach ensures security rules can be regressed-tested quickly during development (`SecurityConfigTest`) while still being validated end-to-end in CI (`SecurityTest`).

3.5 Database and Repository Integration

- **MeetingServiceIntegrationTest**: Performs a complete integration cycle from proposal to database state update, including confirmation logic and iCal token retrieval.
- **MeetingRepositoryTest**: A dedicated `@DataJpaTest` class verifying the custom repository queries `findCalendarMeetings()` (filters declined invites, includes organizer and accepted/pending meetings) and `findOverlapping()` (detects partial, complete, and edge-touching overlaps) in isolation with a test database.

3.6 End-to-End Testing (Selenium)

- **FullFlowE2ETest**: Performs the basic registration → login → propose meeting journey.
- **ExpandedE2ETest**: Verifies discovery searching, meeting creation and visibility in calendar, and logout flows.
- **ICalFeedE2ETest**: Registers a user, creates a meeting, extracts the iCal token from the calendar page, fetches the feed via HTTP, and verifies the VEVENT structure and CONFIRMED status. Also verifies that invalid tokens correctly return 404 (after the fix described in Section 6.1).
- **MeetingResponseE2ETest**: Two-user journey: user1 proposes a meeting inviting user2; user2 accepts and verifies the meeting remains visible; separate test where user2 declines and verifies the meeting disappears from their calendar.

3.7 Test Suite Expansion

After the initial implementation, a critical gap analysis was performed. The following additions were made to strengthen coverage without modifying the SUT:

- **SecurityConfigTest**: Added because **SecurityTest** is a heavy **@SpringBootTest**. A fast unit test allows developers to catch authorization regressions in under a second.
- **AgendaLxProviderTest**: Added as a pure Mockito test because the existing **AgendaLxProviderIntegrationTest** requires a Spring context. Pure unit tests prove the parsing logic without framework overhead.
- **MeetingParticipantTest** and **InviteStatusTest**: Added to complete domain model coverage. Every JPA entity now has dedicated unit tests.
- **MeetingService edge cases**: Added null title, null organizer, empty invitee list, and null status tests to document boundary behavior and prevent silent regressions.
- **AuthController root redirect**: Added to cover the application entry point.
- **ICalFeedE2ETest**: Added because the iCal feed is a key differentiating feature of the application and was not exercised at the browser level.
- **MeetingResponseE2ETest**: Added because the accept/decline flow is core business logic that was never verified through the UI.

The test suite grew from **11 initial classes** to **17+ classes**, achieving comprehensive coverage across all architectural layers.

4 CI/CD and Test Strategy Refactor

To optimize the development lifecycle, the test suite was refactored with a multi-tier execution strategy incorporating real-world CI best practices.

4.1 JUnit 5 Tags

Tests are categorized into five groups:

- **unit**: Isolated logic tests (<< 1s).
- **smoke**: Critical paths required for application stability.
- **integration**: Tests requiring Spring context or database.
- **provider**: External service integration tests (mocked).
- **e2e**: Slow browser automation tests.

4.2 Maven Configuration

The `pom.xml` was updated to support test filtering via properties:

```
1 mvn test -Dgroups="smoke"  
2
```

4.3 GitHub Actions Pipeline

A multi-tier pipeline was implemented in `.github/workflows/ci.yml` following industry-standard practices:

4.3.1 Pipeline Triggers

The workflow runs on three events:

- **Push to any branch:** Triggers fast feedback jobs.
- **Pull Request to main/master:** Triggers integration validation.
- **Manual dispatch** (`workflow_dispatch`): Allows on-demand execution of the full test suite on any branch without requiring a merge.

4.3.2 Concurrency Control

The pipeline uses GitHub's `concurrency` feature to automatically cancel outdated runs when new commits are pushed to the same branch. This prevents wasted runner minutes and ensures only the latest commit is being validated.

4.3.3 Job Structure and Dependencies

- **Fast Tests** (`unit + smoke`): Runs on every push to any branch. Includes a 10-minute timeout and publishes test reports via the Dorny Test Reporter for inline GitHub UI visualization. Artifacts are uploaded for offline debugging.
- **Integration Tests** (`unit + smoke + integration`): Runs only on Pull Requests. Depends on fast tests passing first (`needs: fast-tests`), avoiding unnecessary compute if basic tests fail. 15-minute timeout.
- **Full Tests** (`all groups`): Runs only on pushes to `main` or `master`. Also depends on fast tests. 20-minute timeout. This is the production gate.
- **Manual Full Tests** (`all groups`): Runs exclusively via `workflow_dispatch`. Allows developers to validate the complete suite on a feature branch before opening a PR, without polluting the branch with merge commits.
- **Build JAR:** Runs on every push, packaging the application and uploading the artifact. Proves the app compiles into a deployable artifact.

4.3.4 Real-World Optimizations

- **Job Dependencies:** Slow jobs (integration, full tests) only run if fast tests pass, mimicking the *fail-fast* pattern used in industry pipelines.
- **Timeouts:** Every job has a strict timeout to prevent runaway processes from consuming GitHub Actions minutes.
- **Artifact Uploads:** Test reports and JAR files are preserved as downloadable artifacts, enabling post-mortem analysis without re-running the pipeline.
- **Test Reporter Integration:** JUnit XML results are rendered directly in the GitHub Checks UI, making failures visible without downloading logs.

4.3.5 Running Tests Locally and in CI

```
1 # Run all tests locally
2 mvn test
3
4 # Run only fast tests (feature branch feedback)
5 mvn test -Dgroups="smoke | unit"
6
7 # Run integration-level tests (PR validation)
8 mvn test -Dgroups="smoke | unit | integration"
9
10 # Run full suite (main branch or manual dispatch)
11 mvn test -Dgroups="smoke | unit | integration | provider | e2e"
12
```

5 Modifications to the SUT

To enable effective testing, the following changes were made during the initial implementation:

- **Discovery Providers:** Changed from manual `RestClient` instantiation to `RestClient.Builder` constructor injection. This allows Spring's `@RestClientTest` to intercept and mock HTTP calls.
- **Model Classes:** Added `setId(Long id)` to `User` and `Meeting`. This was necessary to mock persistence-assigned IDs in unit and REST tests.
- **AgendaLxProvider:** Fixed nondeterministic venue selection in `firstVenueName()`. The original implementation used `venue.values().iterator().next()`, which is undefined for unordered `Map` implementations (e.g., `HashMap`, `Map.of()`). The fix sorts keys lexicographically before selecting the first, ensuring deterministic and testable behavior.
- **UserService:** Added explicit null and blank validation for `username`, `email`, and `rawPassword` parameters in `register()`. Previously, null passwords were rejected only incidentally by `BCryptPasswordEncoder`, not by service-level validation.
- **ICalController:** Changed from `@Controller` to `@RestController` and replaced `ResponseStatusException` with direct `ResponseEntity.notFound().build()` return. This fixes the bug where invalid tokens returned 302 (redirect to login) instead of 404 (see Section 6.1).
- **SecurityConfig:** Added `.requestMatchers("/error").permitAll()` to ensure Spring Boot's default error handling is accessible for unauthenticated API endpoints.

6 Known Issues and Bug Analysis

During test execution, several behaviors were identified that deviate from expected outcomes. These are documented below as potential bugs or design gaps, demonstrating the value of comprehensive testing in uncovering latent issues. All **HIGH** and **MEDIUM** severity issues were fixed; **LOW** severity issues are documented for future reference.

6.1 Issue 1: iCal Invalid Token Returns 302 Instead of 404 (FIXED)

Severity: HIGH

Location: `ICalController.feed(String) + SecurityConfig`

Expected: An invalid iCal token should return HTTP 404 (Not Found).

Actual: The E2E test revealed that the full Spring Boot server returns HTTP 302 (Redirect to

/login) for invalid tokens.

Root Cause Analysis: `ICalController` used `@Controller` (MVC view controller annotation) and threw `ResponseStatusException(HttpStatus.NOT_FOUND)`. Spring Boot’s default error handling mechanism intercepts this exception and forwards to the `/error` endpoint. However, `SecurityConfig` did not permit `/error` for anonymous access, so Spring Security’s exception translation filter converted the unauthenticated access to `/error` into a 302 redirect to `/login`. This is a classic interaction bug between Spring Boot’s error handling and Spring Security’s authorization rules.

Impact: API clients (calendar applications) expecting a 404 status code for invalid tokens receive a 302, breaking programmatic error handling and making token validation impossible.

Fix Applied:

1. Changed `ICalController` from `@Controller` to `@RestController` (proper semantic annotation for API endpoints).
2. Replaced `ResponseStatusException` with explicit `ResponseEntity.notFound().build()` return, bypassing Spring Boot’s error handling entirely.
3. Added `.requestMatchers("/error").permitAll()` to `SecurityConfig` as a defensive measure for other potential error-handling scenarios.

Verification: The `ICalFeedE2ETest` was updated to assert `assertEquals(404, response.getStatusCode())` for invalid tokens, and now passes. The `ICalControllerTest` (`@WebMvcTest`) also correctly verifies 404 behavior.

6.2 Issue 2: Nondeterministic Venue Selection in `AgendaLxProvider` (FIXED)

Severity: MEDIUM

Location: `AgendaLxProvider.firstVenueName(Map<String, AlxVenue>)`

Expected: When an event has multiple venues, the provider should consistently return the same venue (e.g., the first by insertion order or key order).

Actual: The method used `venue.values().iterator().next()`, which is nondeterministic for unordered `Map` implementations such as `HashMap` or `Map.of()`. The unit test `search_MultipleVenues_UsesFirst` intermittently failed because the iterator returned the second venue instead of the first.

Impact: Discovery results are unstable — the same query may return different venues for the same event across executions, leading to inconsistent user experience and flaky tests.

Fix Applied: Replaced arbitrary iteration with lexicographic key sorting: `venue.keySet().stream().sorted()`. This guarantees deterministic “first” selection based on the venue ID keys returned by the API.

Recommendation: Always assume `Map` iteration order is undefined unless using `LinkedHashMap` or `SortedMap`. Document ordering assumptions in method contracts.

6.3 Issue 3: Null Password Handling in `UserService` (FIXED)

Severity: MEDIUM

Location: `UserService.register(String, String, String)`

Expected: Registering a user with a null password should throw a clear `IllegalArgumentException` with a descriptive message.

Actual: The service accepted null passwords silently. The `PasswordEncoder` mock returns `null` when `encode(null)` is called, and the `User` constructor stores this null value without validation.

Impact: Users could be created with null password hashes, making authentication impossible and creating invalid database state. While the real `BCryptPasswordEncoder` throws `IllegalArgumentException` on `encode(null)`, this is “accidental safety” not “intentional safety.”

Fix Applied: Added explicit null/blank checks in `UserService.register()` before encoding. This makes the service robust regardless of which `PasswordEncoder` implementation is wired in.

Verification: `UserServiceTest` now includes `register_NullPassword_ThrowsIllegalArgumentException`, `register_EmptyPassword_ThrowsIllegalArgumentException`, and `register_BlankPassword_ThrowsIllegalArgumentException` all passing.

6.4 Issue 4: AgendaLx API Returns Non-Standard JSON for Venues (DOCUMENTED)

Severity: LOW (external dependency)

Location: `AgendaLxProvider` parsing logic

Expected: The `venue` field in the AgendaLx API response is consistently a JSON object mapping venue IDs to venue details.

Actual: The E2E discovery test revealed that the live API sometimes returns `false` (boolean) instead of an object for the `venue` field, causing Jackson deserialization to fail with `MismatchedInputException`. The provider's catch block swallows this and returns an empty list.

Impact: When the API returns malformed venue data, the entire search query fails and returns an empty list, even if other events are valid.

Status: Not fixed — this is an external API behavior outside our control. The current graceful degradation (returning empty list) is acceptable for a “nice to have” discovery feature.

Recommendation: Make the `venue` field deserialization more lenient (e.g., using a custom deserializer that handles both object and boolean types), or wrap the per-event parsing in a try-catch to skip individual malformed events rather than failing the entire batch.

7 How to Run the Tests

Ensure you have Maven and a JDK (17+) installed.

1. Navigate to the `calendar` directory.
2. Run all tests: `mvn test`
3. To run a specific test class: `mvn test -Dtest=ClassName`
4. To run by tag: `mvn test -Dgroups="smoke | unit"`

8 Conclusion

The resulting test suite provides high coverage across all application layers, growing from 11 to 17+ test classes. The combination of fast unit tests (including the pure-Mockito `AgendaLxProviderTest` and `SecurityConfigTest`), comprehensive integration tests, and thorough E2E tests ensures that both business logic and user-facing features are reliable. The dual-layer security testing strategy (fast `@WebMvcTest` + full `@SpringBootTest`) provides rapid regression feedback without sacrificing end-to-end confidence.

Importantly, the testing effort uncovered and **fixed five concrete bugs** in the application: (1) **HIGH** invalid iCal tokens returning 302 instead of 404 due to Spring Security/Spring Boot error handling interaction, (2) **MEDIUM** nondeterministic venue selection in `AgendaLxProvider`, (3) **MEDIUM** missing null password validation in `UserService` and one additional external-API issue, (4) (non-standard JSON venues) was documented but not fixed as it is outside application control.

These findings demonstrate that a well-designed test suite is not merely a verification tool, but an active instrument for quality assurance and defect detection. The CI pipeline follows real-world conventions with staged execution, manual triggers, concurrency control, and artifact

management, making the development workflow efficient and maintainable. The entire expansion was achieved with minimal SUT modifications, validating the testability of the original architecture.